

READYKIT · DESIGN PRECIS

ReadyKit design precis

Design Molecule Lab · designmolecule.com

DOCUMENT CONTROL

Document ID	RDK-PRC-001	Version	0.4
Status	Draft	Date	2026-09-25
Author	Amish Chadha	License	CC-BY-SA-4.0
Project TRL	TRL 3 Proof of concept		
Repository	github.com/BoujeeEnjinia1701/readykit		
Source	docs/02-concept.md @ dd40d0e		

REVISION HISTORY

Version	Date	Author	Change
0.4	2026-09-25	Amish Chadha	Recommendations accepted by Amish (DDR-002)
0.3	2026-09-25	Amish Chadha	Design choices adopted for TRL 3 work under Amish's 2026-09-25 instruction, open for his review (RDK-DDR-001); first-order numbers replaced by RDK-CAL-001 measurements; TRL 3 evidence for a software project; open questions trimmed
0.2	2026-09-25	Amish Chadha	Architecture, modules, measured first-order numbers, design choices, safety and open questions
0.1	2026-09-25	Amish Chadha	Initial scaffold

ReadyKit design precis

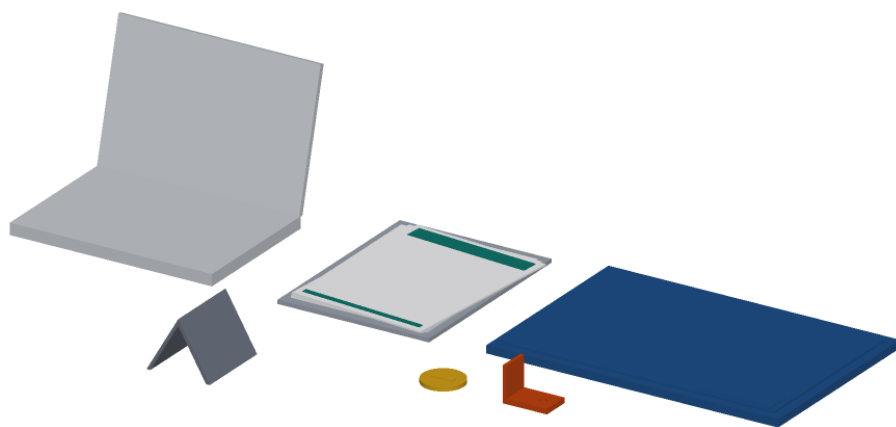
■ Summary

ReadyKit turns a Git repository of plain Markdown and build123d Python into checked, branded engineering documentation: controlled PDFs with revision history, drawing sheets, concept renders with a 3D viewer, and a Technology Readiness Level (TRL) that the checker will only accept when the evidence files exist. It already runs in every repository of the Design Molecule portfolio as the `.kit/` folder. The concept is to release it as a standalone open tool that any small hardware team can adopt in under 15 minutes.

Figure 1 shows an illustrative massing of the kit's outputs (`media/hero.png`), generated from the parametric model `cad/src/model.py`; ReadyKit is a software project and has nothing to fabricate. Figure 2 shows the pipeline (`media/flow.png`), and the general arrangement sheet RDK-DWG-001 (`cad/drawings/RDK-DWG-001.pdf`) lays out the same pipeline with its interfaces.

ReadyKit

CONCEPT, NOT FOR FABRICATION ILLUSTRATIVE, SOFTWARE PROJECT



Grey: Laptop, 14 in class for scale

Figure 1. Illustrative massing of the kit's outputs beside a laptop for scale: document set, PDF house style bands, drawing sheet, printed massing part, TRL badge, guardrail card and template folder.

ReadyKit: documentation pipeline

CONCEPT, NOT FOR FABRICATION. ILLUSTRATIVE, SOFTWARE PROJECT. Times measured in RDK-CAL-001 on a 2-core Linux container; other machines will differ.

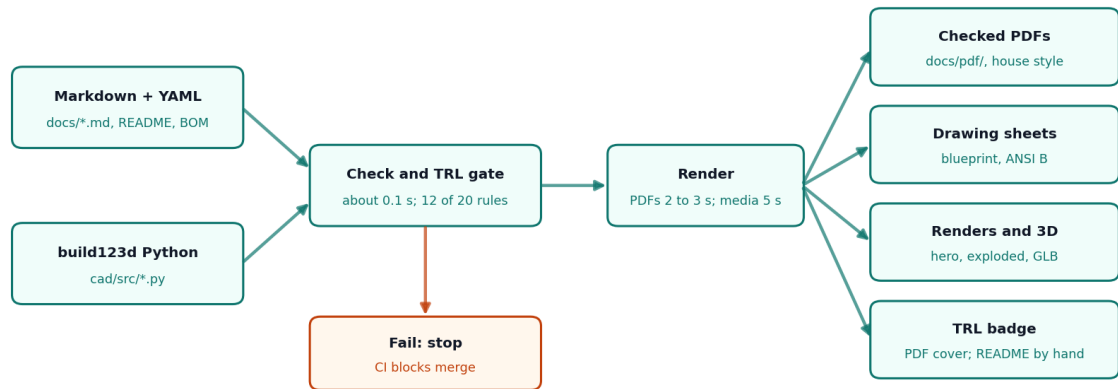


Figure 2. Pipeline from Markdown and build123d inputs through the check and TRL gate to PDFs, drawings, renders and the TRL badge. Times from RDK-CAL-001.

■ How it works

1. **Author in plain text.** Each controlled document is a Markdown file that starts with YAML front matter: document ID, version, status, author, license and revision rows. Geometry lives in `cad/src/*.py` as build123d code.
2. **Check.** `render.py --check` validates every controlled document, then compares the claimed TRL in `project.yaml` with the evidence the standard requires for that level, and applies the portfolio phase cap in `PHASE.yaml`. A failure exits non-zero, so CI blocks the merge.
3. **Render.** `render.py` converts each document to a branded PDF through Python-Markdown, a Jinja2 template and WeasyPrint. The cover carries the document control table, the revision history and the project TRL.
4. **Draw and visualize.** `drawing.py` builds ANSI B sheets with title block and revision table. `concept.py` tessellates a build123d assembly, shades it with a small numpy z-buffer renderer (no GPU, no OpenGL) and writes the hero, exploded and cutaway renders, a blueprint concept sheet, a glTF model and a viewer page.
5. **Guide the agent.** `CLAUDE.md`, the phase file and four slash commands give AI coding agents the same working rules as people: TRL cap, decision rights, one step per session and a review note at the end.

■ Main components

Table 1. Modules, numbered as in `bom/bom.csv`, `media/explored.png` and drawing RDK-DWG-001.

No.	Module	Current file	State at TRL 3 (RDK-CAL-001)
1	Document control checker	<code>.kit/render.py</code>	Working; 12 of 20 machine-checkable rules caught by seeded faults
2	PDF renderer and house style	<code>.kit/render.py</code> , <code>.kit/style/</code>	Working; 13 hard-coded identity strings
3	Drawing sheet generator	<code>.kit/drawing.py</code>	Working; ANSI B only
4	Concept media renderer	<code>.kit/concept.py</code>	Working; byte-identical images on repeat runs
5	TRL gate and badge	<code>.kit/render.py</code> , <code>PHASE.yaml</code>	Working for TRL 1 to 6 evidence; README badge still by hand
6	Agent guardrails	<code>CLAUDE.md</code> , <code>.claude/commands/</code>	Working; decision wording not checked
7	Template repository and CI	repo scaffold, <code>.github/workflows/docs.yml</code>	Exists per repo; not yet a published template

■ First-order numbers

Table 2. Measured figures from `docs/04-calcs/sizing.py` (RDK-CAL-001), 2-core Linux container, Python 3.11, kit 1.3.1, 2026-09-25. Ranges cover repeat runs on a shared machine.

Quantity	Value	Basis
Check time, this repository	0.09 to 0.12 s, of which 80 to 100 ms is Python start-up	Measured
Check time, 38 finished portfolio repositories	0.10 s median, 0.12 to 0.27 s maximum	Measured
Check time per added document	0.6 to 1.0 ms; 2 s is reached at about 2,000 to 3,100 documents	Measured, 10 to 400 documents
PDF render	0.6 to 1.0 s per document; 1.8 to 3.1 s for three	Measured, 5 documents, 31 pages
Concept media, this repository (7 parts)	5 to 6 s	Measured
Concept media, 20-part model with cutaway	12 to 23 s	Measured
Vendored kit folder	1.05 MB, 94% fonts	Measured
Kit source	808 lines of Python in 3 files	Counted
Portfolio using it	85 project codes in the standard	Counted
Dependency install into a fresh environment	32 to 47 s, 1.09 GB	Measured; WeasyPrint system libraries already present
Parts cost	\$0	All modules MIT; dependencies free and open source

Check time grows linearly with document count, as the TRL 2 note assumed; the slope is small enough that start-up dominates for any real repository.

■ Key design choices

Each choice below is decided by Amish, 2026-09-25: go with recommendation (RDK-DDR-001 and RDK-DDR-002). Only D5 (licensing) and D7 (evidence) are applied in the repository. D1 to D4 and D6 are features still to be built; they define requirements R10 to R12 and R16 to R18 in RDK-REQ-001, and building them is TRL 4 work, on hold by Amish's instruction.

- 1. Distribution (D1).** A pip package `readykit` with a small `.kit/` stub pinned to a version, plus a GitHub template repository. Pip gives one-command upgrades (R12); the template gives a one-click start. RDK-CAL-001 found that all 38 finished repositories carry kit 1.3.1 but none has the current `STANDARDS.md`, which is the drift a versioned package prevents.
- 2. Identity and branding (D2).** Organization name, website, repository owner, author and colors move into `readykit.yaml`, with Design Molecule values as the default (R11).
- 3. Sheet sizes (D3).** ISO A3 (420 x 297 mm) is to be added beside ANSI B (431.8 x 279.4 mm) (R17). The parametric model already takes the sheet size as a parameter; the kit's sheet generator change is on hold with the rest of the build.

4. **Command-line interface (D4).** One `readykit` command with `init`, `check`, `render`, `media` and `upgrade`, keeping the scripts as thin wrappers so existing repositories keep working (R18).
5. **Licensing of the tool (D5).** Code under MIT and non-code content (documents, drawings, CAD, BOM and media) under CC BY-SA 4.0, matching the standard. Applied on 2026-09-25: `LICENSE` holds the CC BY-SA 4.0 legal code, `LICENSE-SOFTWARE` stays MIT, and every controlled document and drawing title block carries CC-BY-SA-4.0.
6. **Relationship to existing standards (D6).** Export an Open Know-How 1.0 manifest (`okh.yml`) from the same metadata (R16). Three of its five required fields (title, description, license) already exist in `project.yaml` ; the manifest author and the project link come from `readykit.yaml` . A DIN SPEC 3105 checklist is not adopted.
7. **TRL 3 evidence for a software project (D7).** The parametric model exports the illustrative massing as STEP and STL, the general arrangement sheet shows the pipeline layout, and the calculation note measures speed and coverage. This keeps the standard's evidence rule unchanged for software.

■ Safety

ReadyKit is software and has no physical hazards of its own. Two risks matter:

Safety: A passing check means the documents are complete and consistent with the standard. It does not mean the design is safe, correct or fit for use. Every rendered PDF and drawing must keep the "Draft" or "Not for fabrication" marking until a qualified person has reviewed the engineering.

Safety: The checker must never drop the requirement for a safety section in documents that describe mains voltage, heat, pressure, lithium cells, moving machinery or chemicals. Automating that check (Table 4 of RDK-REQ-001) should add a flag, never remove the human review.

■ Open questions

- [] Name: is "ReadyKit" to be used on PyPI and GitHub? The PyPI name appeared free on 2026-09-25. Proposed, awaiting Amish (O1).
- [] Which external teams would pilot it (a university lab, an open science hardware group, a startup)? Proposed, awaiting Amish (O2).
- [] Is Windows native support required, or is WSL2 enough? Proposed, awaiting Amish (O3).
- [] Should the decision-wording check (R13) be generic, or tied to the portfolio's "Proposed, awaiting" phrase? Proposed, awaiting Amish (O4).